Указатели

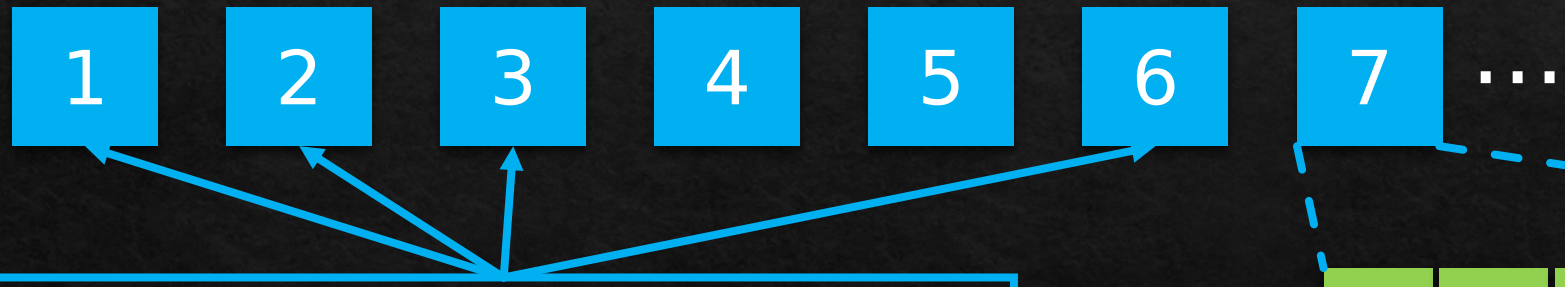
Линейная память, адресация, указатель

Линейная память

Бит, байт, однобайтовые и многобайтовые данные

Линейная память

Подавляющее большинство современных компьютеров работают с линейной организацией памяти. Такой подход рассматривает память как непрерывный набор элементов данных, каждому из которых присвоен свой уникальный номер.



Минимальный адресуемый элемент называется байт. Каждый байт в линейной памяти имеет свой уникальный номер.

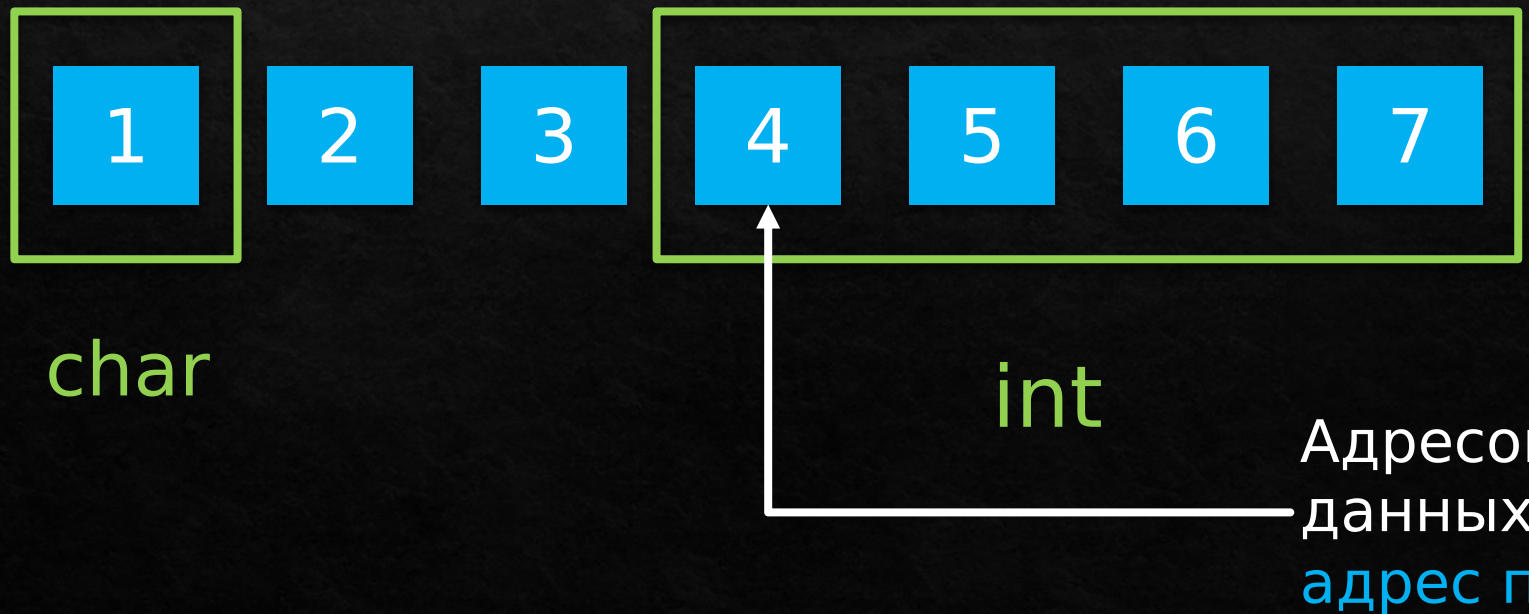


Отдельные байты состоят из битов. В подавляющем большинстве случаев в современных системах 1 байт содержит 8 бит. Специальная константа CHAR_BIT указывает количество бит в байте на вашей платформе.

Многобайтовые данные

Разные типы данных в C++ могут состоять из различного числа байтов. Например типы char / unsigned char / signed char – состоят строго из одного байта. В этом случае ясно, что каждая переменная этого типа располагается в своём уникальном месте в памяти.

Если мы рассмотрим более сложные данные, например тип int, который, предположим, занимает 4 байта, то ему будут соответствовать сразу 4 места в памяти!



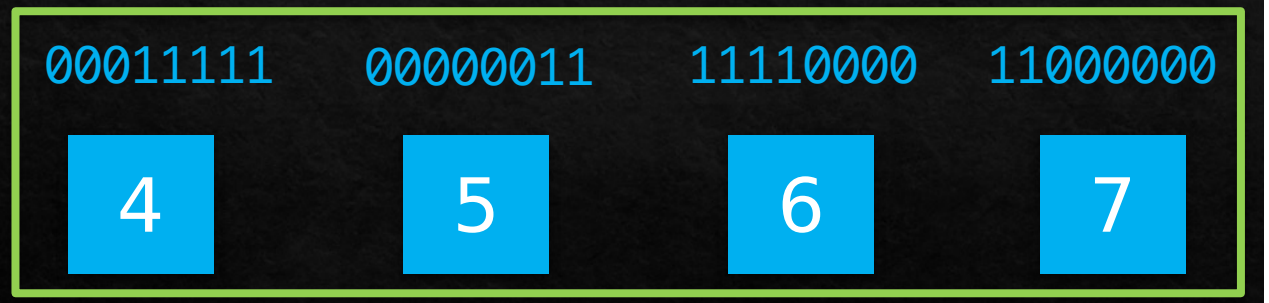
Многобайтовые данные - endian



Если наше число состоит из нескольких байтов, то возникает вопрос: а в каком порядке эти байты хранятся относительно побитовой записи числа?

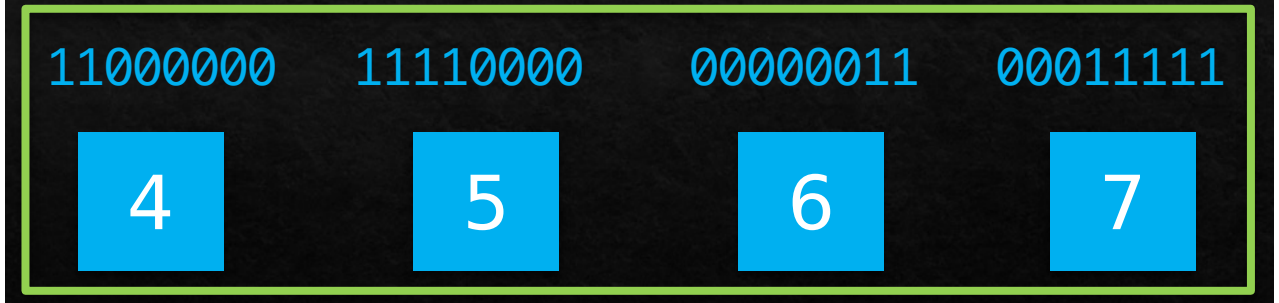
`00011111 00000011 11110000 11000000` **(520’351’936)**

Big endian



Big endian располагает младшие разряды в старших адресах, такой порядок соответствует чтению числа побитово: следующий бит в числе – это следующий бит в том же байте или первый бит в следующем байте.

Little endian



Little endian располагает младшие разряды в младших адресах, но порядок битов при этом не меняется. Такой подход позволяет эффективнее организовать преобразование типов.

Многобайтовые данные – Little endian



В случае little endian различные типы данных, получаемые отбрасыванием старших разрядов, имеют один и тот же адрес.

Многобайтовые данные

Различные системы могут использовать различные представления многобайтовых данных. Это может оказаться критическим при переносе двоичных данных с одной платформы на другую, например по сети.

Проблемам переноса данных подвержены, например двух- и четырёх-байтовые кодировки текста UTF-16 и UTF-32 соответственно. В некоторых случаях в начало таких текстовых файлов добавляют специальные данные, определяющих порядок байтов – Byte Order Mark (BOM).

0xFEFF – big endian UTF-16
0xFFFE – little endian UTF-16

Адресация

Операция взятия адреса &, преобразование адреса в число

Взятие адреса

Если в вашей программе определена переменная, то с этой переменной связано некоторое место в памяти, где и будут храниться её данные. С помощью специальной операции унарного & мы можем получить номер того места, где расположена наша переменная.

```
6       int i = 5;
7       std::cout << i << " " << &i << std::endl;
```

```
5 0x7ff7b708250c
```

В переменной находится число 5. Длинное же 8 байтовое число (ведущие нули опущены) – это номер первого из 4 байт, которые выделены этой переменной для хранения данных.

Операция взятия адреса возвращает значение соответствующее местоположению первого байта переменной i.

Взятие адреса

Так как в C++ можно перегружать операторы, то оператор взятия адреса тоже может быть перегружен. Чтобы получить адрес вне зависимости от перегрузки оператора `&`, используйте функцию `std::addressof`, определённую в заголовочном файле `memory`.

```
#include <memory>
```

```
18      int i = 5;
19      std::cout << i << " " << std::addressof(i) << std::endl;
```

```
5 0x7ff7b708250c
```

Функция для взятия адреса вне зависимости от перегруженного оператора `&`.

Взятие адреса

Адрес можно взять даже у неинициализированной переменной. Хотя сама переменная не имеет конкретного значения, она уже расположена в памяти и ей будет задан конкретный адрес.

```
27      int i;
28      std::cout << &i << std::endl;
```

```
0x7ff7b091050c
```

От временного значения нельзя взять адрес.

```
36      int i = 5;
37      std::cout << &(i + 2) << std::endl;
```

```
error: cannot take the address of an rvalue of type 'int'
std::cout << &(i + 2) << std::endl;
             ^ ~~~~~
```

Взятие адреса

Адрес, по существу, является целым числом, порядковым номером байта, с которого начинается размещение данных. Однако хранить эти данные следует в специальных переименованиях: `uintptr_t` и `intptr_t`, потому что конкретный размер данных, в которых хранятся адреса, зависит от платформы. Эти переименования определены в заголовочном файле `cstdint`.

Для преобразования адреса в число используйте `reinterpret_cast`.

```
47      int i = 5;
48      uintptr_t address = reinterpret_cast<uintptr_t>(&i);
49      std::cout << i << " " << &i << " " << address << std::endl;
```

```
5 0x7ff7b5b3a50c 140701882098956
```

Виртуальная память

Современное оборудование и операционные системы (ОС) позволяют процессам использовать гораздо больше памяти, чем на самом деле есть физической памяти в наличии, если процесс не использует всю память одновременно. Часть данных процесса может быть временно сохранена в энергонезависимой памяти. Такой подход называют виртуальной памятью. Адреса, которыми мы оперируем в наших программах, называются виртуальными, они не являются физическими адресами, их отображением занимается ОС.

Подробно о различных вариантах организации памяти читайте в [1].

[1] В.Е. Карпов К.А. Коньков <<Основы операционных систем Курс лекций>>, издание 3-е, М.: Физматкнига, 2019

Указатель

Объявление указателей, особенности объявления указателей, сохранение адреса, инвариантность связанного типа

Указатель

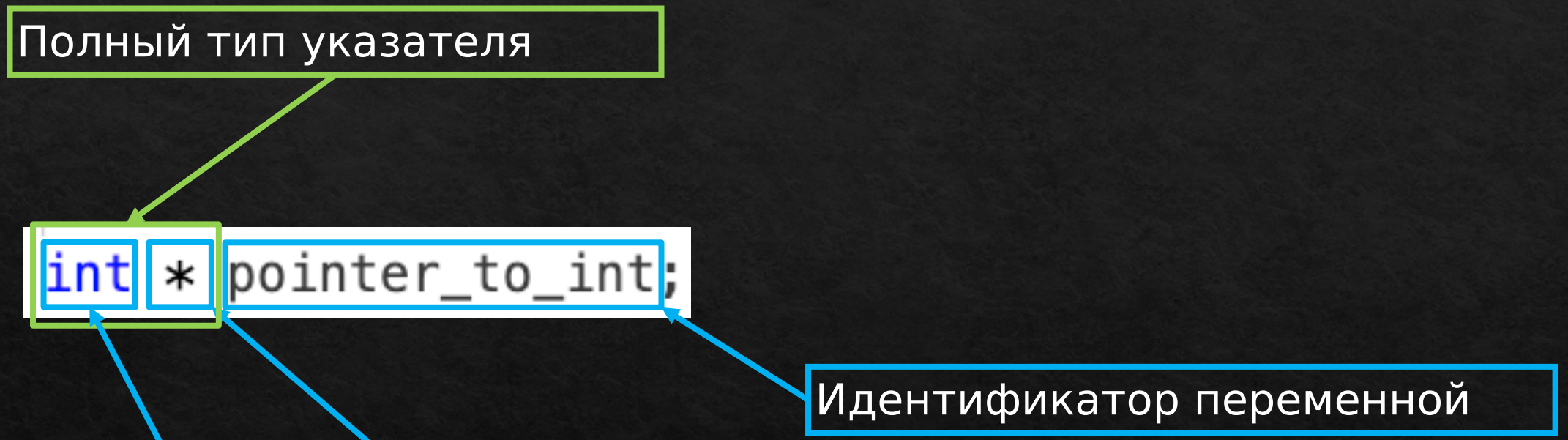
Указателем называют специальную переменную (и тип данных) для хранения адресов.

Специальный тип данных для работы с адресами нужен для удобного и безопасного получения данных по их адресам.

Указатель позволяет:

- специальную операцию «разыменвоание» - получение данных по адресу;
- только явное приведение указателя из одного типа в другой;
- арифметические операции с указателем для быстрого вычисления адресов смежных данных;
- специальный тип указателя для хранения адреса, для которого тип связанных данных неизвестен.

Объявление переменной-указателя



Связанный тип данных необходим указателю для правильного чтения из памяти. Указатель хранит только один адрес – адрес первого байта данных, так как данные могут состоять из нескольких байт, то по связанному типу компилятор понимает, сколько именно следует прочитать байт и как интерпретировать полученные байты. Например int и float содержат одинаковое количество байт в 32-битных системах, но числа представлены в них по-разному.

Особенности объявления указателей

```
// два указателя
int *a_ptr, *b_ptr;

// указатель и целое число
int *i_ptr, i;

// целое число и указатель
int j, *j_ptr;

/**
 * Символ '*' может стоять и возле типа данных, но
 * указателем будет только первый идентификатор!
 */
int* q_intptr, q_int;
```

Обратите внимание! Символ ‘*’ может стоять в разных местах! В некоторых случаях это может оказаться важным.

Особенности объявления указателей

```
69  #include <typeinfo>
70
71  typedef int* iptr_t;
72
73  void typedef_pointer_example() {
74      // два указателя
75      iptr_t a, b;
76      std::cout
77          << typeid(a).name() << " "
78          << typeid(b).name() << std::endl;
79  }
80
81  using uiptr_t = unsigned*;
82
83  void using_poiter_example() {
84      // два указателя
85      uiptr_t a, b;
86      std::cout
87          << typeid(a).name() << " "
88          << typeid(b).name() << std::endl;
89  }
```

Отдельно стоит упомянуть возможность переименования типов, при которой все, а не только первый, идентификаторы при объявлении окажутся указателями.

Сохранение адреса

Взяв адрес от некоторой переменной, его можно сохранить в указателе с таким же связанным типом.

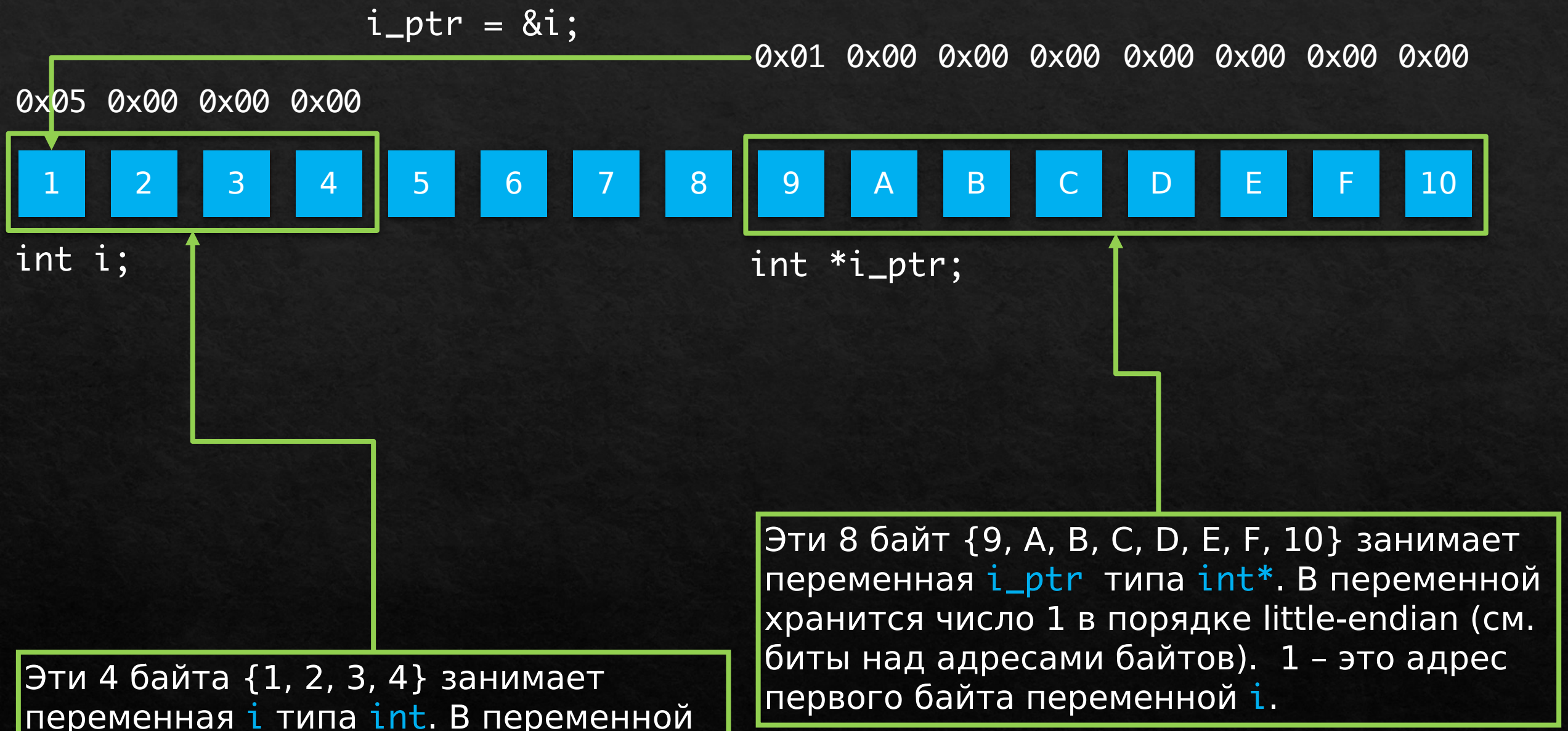
```
int i;
int *i_ptr = &i;

unsigned j;
unsigned *j_ptr = &j;

float f;
auto *f_ptr = &f;

long l;
auto l_ptr = &l;
```

Ключевое слово auto позволяет компилятору автоматически вывести связанный тип.


i_ptr = &i;
0x01 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x05 0x00 0x00 0x00
1
2
3
4
5
6
7
8
9
A
B
C
D
E
F
10
int i;
int *i_ptr;
Эти 4 байта {1, 2, 3, 4} занимает переменная i типа int. В переменной хранится число 5 в порядке little-endian (см. биты над адресами байтов).
Эти 8 байт {9, A, B, C, D, E, F, 10} занимает переменная i_ptr типа int*. В переменной хранится число 1 в порядке little-endian (см. биты над адресами байтов). 1 – это адрес первого байта переменной i.

Инвариантность связанного типа

Хотя для типов данных и допускаются неявные преобразования и в little-endian представление сужающее преобразование целочисленных типов даже не меняет адрес данных, неявное преобразование указателей запрещено.

```
112     // неявные преобразования между типами
113     int   i = s;
114     long  l = i;
115
116     // неявные преобразования указателей запрещены
117     int *i_ptr = &s;
118     long *l_ptr = &i;
```

```
Pointers/pointers.cpp:117:10: error: cannot initialize a variable of type 'int *' with an rvalue of type 'short *'
    int *i_ptr = &s;
         ^       ~~
Pointers/pointers.cpp:118:11: error: cannot initialize a variable of type 'long *' with an rvalue of type 'int *'
    long *l_ptr = &i;
          ^       ~~
```

Разыменование

Разыменование, косвенная модификация

Разыменование (косвенное обращение)

Разыменование – это специальная операция, которая позволяет по адресу получить данные. Так как указатель содержит адрес только первого байта, то для получения данных необходим связанный тип. Кроме того, необходимо понимать как именно интерпретировать полученные данные (например int или float).

Для этой операции в C++ есть специальный оператор – унарный символ ‘*’, который ставится перед указателем.

Объявление указателя

```
int i = 5;
int *i_ptr = &i;
std::cout << *i_ptr << std::endl;
```

Оператор разыменования

Приоритет	Оператор	Описание
1	::	Разрешение области видимости
2	a++ a--	Суффиксный/постфиксный инкремент и декремент
	тип() *тип*{}	Функциональный оператор приведения типов
	a()	Вызов функции
	a[]	Индексация
	. ->	Доступ к элементу
3	++a --a	Префиксный инкремент и декремент
	+a -a	Унарные плюс и минус
	! ~	Логическое НЕ и побитовое НЕ
	(*тип*)	Приведение типов в стиле C
	*a	Косвенное обращение (разыменование)
	&a	Взятие адреса
	sizeof	Размер в байтах
	co_await	Выражение await (C++20)
	new new[]	Динамическое распределение памяти
	delete delete[]	Динамическое освобождение памяти

Косвенная модификация данных

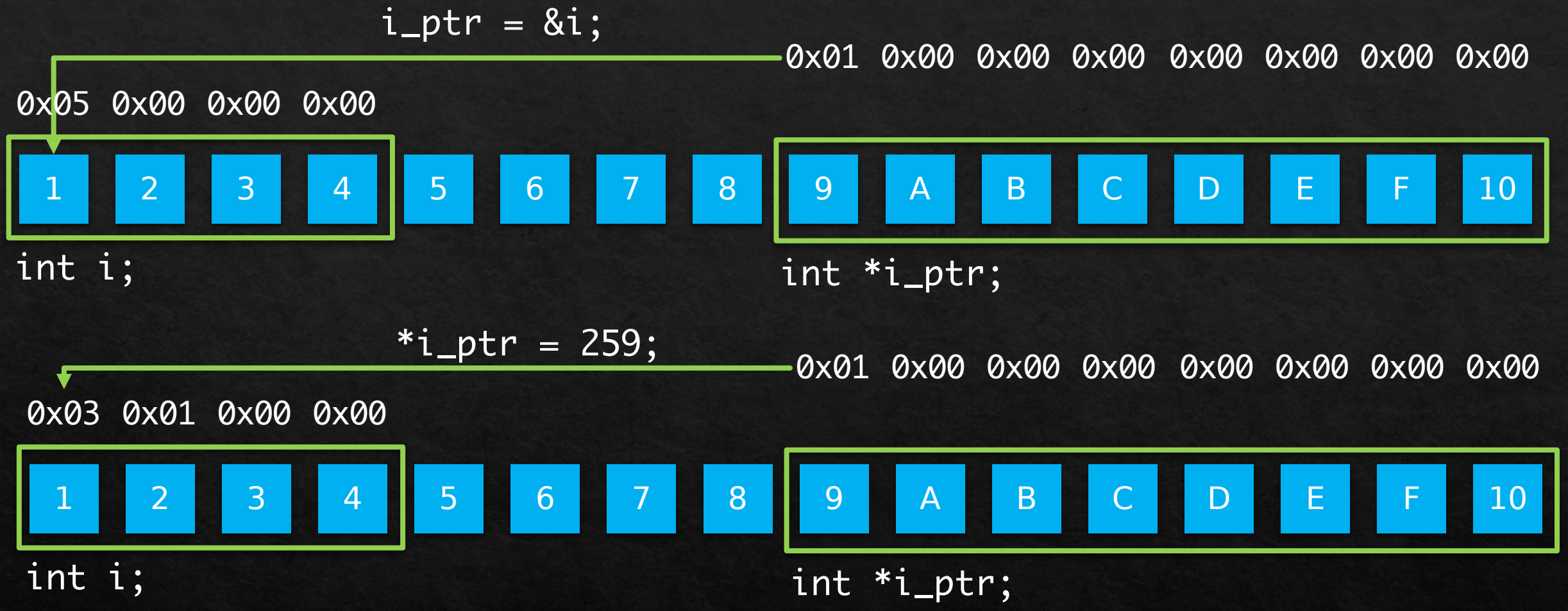
```
138     int i = 5, *i_ptr = &i;
139
140     std::cout << "Prev: " << i << '\n';
141
142     *i_ptr = 7;
143
144     std::cout << "New: " << i << '\n';
```

Объявление указателя и целого числа

Косвенная модификация

```
Prev: 5
New: 7
```

При разыменовании вы получаете доступ к данным, как если бы вы работали с самой переменной.



Расположение данных в памяти не меняется, но содержимое 4 байт, принадлежащих переменной `i`, изменилось посредством указателя `i_ptr`.

Указатели и константность

Неизменяемый указатель и указатель на неизменяемые данные

Сохранение адреса константы

Так как посредством указателя можно менять данные, то мы должны ввести ограничения для модификации констант. Это сделано на уровне проверки типов. Указатели на переменные и указатели на константы имеют различный тип.

```
int const cv_i = 42;
int *i_ptr = &cv_i;
```

Адрес константы

Указатель на переменную

```
error: cannot initialize a variable of type 'int *' with an rvalue of type 'const int *'
```

Адрес константы получает тип `int const *`, т.е. другой связанный тип.

Разные связанные типы!

Сохранение адреса константы

Объявляем указатель со связанным типом `int const`.

```
int const cv_i = 42;

int const *i_ptr;

i_ptr = &cv_i;

std::cout << *i_ptr << std::endl;
```

Указатель с таким связанным типом говорит нам: «При разыменовании я превращаюсь в неизменяемый int». Такой указатель даёт возможность косвенно читать данные, но не модифицировать их! Его называют указатель НА константу или указатель НА неизменяемые данные.

```
*i_ptr = 7;
```

```
error: read-only variable is not assignable
```

```
*i_ptr = 7;
~~~~~~ ^
```

Изменяемый и неизменяемый указатели

Как и любую другую переменную мы можем изменять указатель, менять адрес, который сохраняем.

```
int i = 5, j = 7;

int *ptr = &i; // инициализация
std::cout << *ptr << '\n';

ptr = &j; // присваиваем новый адрес
std::cout << *ptr << '\n';
```

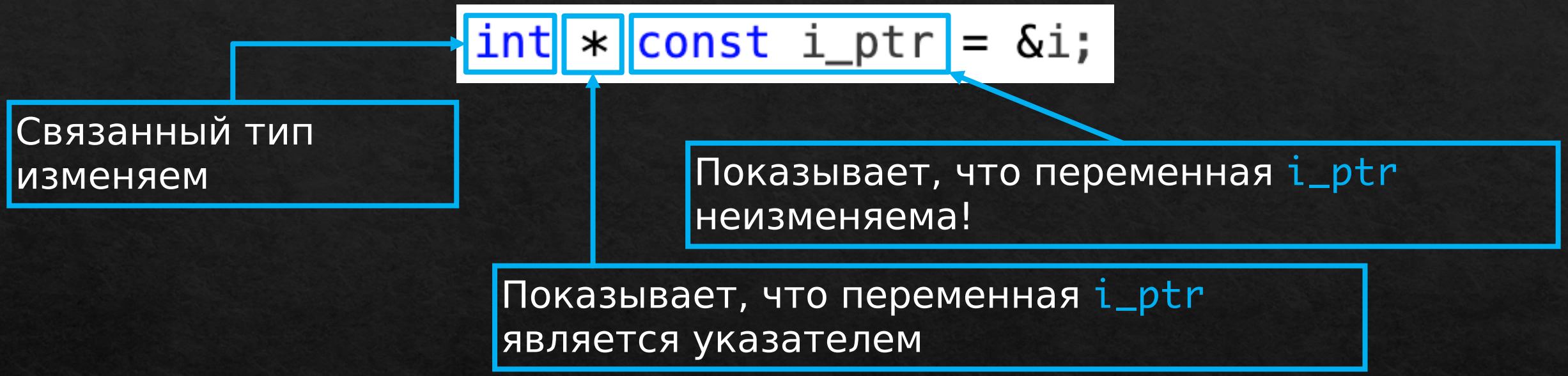
Если мы не хотим изменять указатель, то нам нужно объявить сам указатель неизменяемым.

```
int i = 5, j = 9;
std::cout << i << '\n';

int * const i_ptr = &i;
std::cout << *i_ptr << '\n';

*i_ptr = 7;
std::cout << i << '\n';
```

Неизменяемый указатель




```
int * const i_ptr = &i;
```

```
i_ptr = &j;
std::cout << *i_ptr << '\n';
```

```
error: cannot assign to variable 'i_ptr' with const-qualified type 'int *const'
```

Хотя мы можем менять данные по адресу, сохранённому в i_ptr, мы не можем изменить сам i_ptr, не можем поменять сохранённый адрес.

Неизменяемый указатель на неизменяемые данные



Указатели для косвенного обращения

В языке C, в отличие от C++, нет ссылок и единственным способом косвенного обращения к данным остаются указатели.
Ниже представлен пример обмена данными между двумя числами типа int посредством указателей.

```
228  void swap_with_pointers(int * const a, int * const b) {
229      int tmp = *a;
230      *a = *b;
231      *b = tmp;
232  }
233
234  void swap_with_pointers_test() {
235      int x = 1, z = 5;
236      std::cout << "(x,z) = " << x << " " << z << '\n';
237
238      swap_with_pointers(&x, &z);
239      std::cout << "(x,z) = " << x << " " << z << '\n';
240
241      std::cout << std::endl;
242  }
```

```
(x,z) = 1 5
```

```
(x,z) = 5 1
```

nullptr

Нулевой указатель

В некоторых ситуациях мы хотим, чтобы указатель имел некоторое корректное значение, но не был связан с данными. Это особое состояние указателя – нулевой указатель.

В языке C используется специальная макро-константа NULL. Она также определена и в C++ в заголовочном файле cstddef. Само значение NULL может быть не только нулём, это зависит от платформы.

Начиная с C++11 в C++ определено специальное ключевое слово nullptr. Это особая константа, которая имеет специальный тип std::nullptr_t. Такое нововведение было необходимо для разрешения проблемы перегрузки функций. Дело в том, что NULL – это макроопределение, которое скрывает некоторую числовую константу, которая свободно может проходить в перегруженные для целых чисел функции.
Ещё одна особенность nullptr – это возможность неявного преобразования в любой другой тип указателя.

```
286     {
287         auto null_ptr = NULL;
288         int *ptr = null_ptr;
289     }
290
291     {
292         auto null_ptr = nullptr;
293         int *ptr = null_ptr;
294     }
```

```
Pointers/pointers.cpp:288:14: error: cannot initialize a variable of type 'int *' with an lvalue of type 'long'
        int *ptr = null_ptr;
             ^     ~~~~~~~~
```

```cpp
void overload_f(long l) {
    std::cout << "Long: " << l << std::endl;
}

void overload_f(int *ptr) {
    std::cout << "Pointer: " << ptr << std::endl;
}

void overload_with_null_nullptr() {
    overload_f(NULL);
    overload_f(nullptr);
}
```

```
Long: 0
Pointer: 0x0
```

Разыменование нулевого указателя

Язык C (C99) строго запрещает разыменование нулевого указателя в любых обстоятельствах. Это поведение не определено (`undefined behavior`).

В языке C++ ситуация немного сложнее. Хотя прямо стандарт не запрещает разыменование нулевого указателя, в различных местах стандарта описываются ситуации неопределённого поведения с участием `nullptr`. Например, неопределённым является попытка вызвать метод объекта по нулевому адресу, однако для статического метода такой вызов уже будет допустим.

Наиболее простая стратегия – придерживаться правила, что разыменование нулевого указателя ведёт к неопределённому поведению.

Указатели
vs
ссылки

Переменные-указатели содержат собственную память.

Размер ссылки – это размер связанного типа, у ссылки нет своей памяти.
Размер указателя – это размер памяти, необходимой для сохранения адреса.

```
char c = 'a';
char &c_ref = c;
char *c_ptr = &c;
std::cout << sizeof(c) << " " << sizeof(c_ref) << " " << sizeof(c_ptr) << '\n'; 1 1 8

int i = 5;
int &i_ref = i;
int *i_ptr = &i;
std::cout << sizeof(i) << " " << sizeof(i_ref) << " " << sizeof(i_ptr) << '\n'; 4 4 8
```

Указатели можно оставлять неинициализированными.
Указатели можно присваивать друг другу, связывать указатель с новыми данными.

```
int i = 5, j = 7;
int *ptr; // не инициализируем
ptr = &i; // запоминаем адрес
std::cout << *ptr << '\n';

ptr = &j; // запоминаем другой адрес
std::cout << *ptr << '\n';
```

Ссылки могут быть связаны с временными данными.

```
270  void reference_to_rvalue(int const &ref) {
271      std::cout << ref << std::endl;
272  }
```

```
274  void pointer_to_rvalue(int const *ptr) {
275      std::cout << *ptr << std::endl;
276  }
```

```
int i = 5, j = 7; // (i + j) -> rvalue // временное значение

reference_to_rvalue(i + j);
pointer_to_rvalue(&(i + j));
```

```
12
error: cannot take the address of an rvalue of type 'int'
pointer_to_rvalue(&(i + j));
                  ^ ~~~~~
```

Для косвенного обращения к данным предпочитайте ссылки, они лучше выражают абстракцию псевдонима.

void*

Адрес без связанного типа

В некоторых ситуациях возникает необходимость передать адрес данных, не зная точно какие данные и какого размера хранятся по этому адресу, подразумевая, что тот, кому вы передаёте этот адрес знает, что делать.

Две наиболее часто возникающие ситуации: получение неинициализированной памяти и передача параметров, детали которых скрыты.

В этих случаях применяется специальный указатель `void*`, это указатель без связанного с ним типа. Он сохраняет любой адрес, в него неявно может быть преобразован любой указатель.

```
void *ptr;

int i = 5;
ptr = &i;

unsigned u = 7u;
ptr = &u;
```

Хотя любой указатель можно преобразовать в `void*`, обратное преобразование неявно запрещено в C++. В языке же C такое обратное преобразование также разрешено неявно.

```
int *i_ptr = ptr;
```

```
: error: cannot initialize a variable of type 'int *' with an lvalue of type 'void *'
```

Явное привидение типов указателей

reinterpret_cast, strict aliasing, std::byte

reinterpret_cast

Для явного приведения указателей одного связанного типа к другому в C++ используется `reinterpret_cast`. Особенностью `reinterpret_cast` является отсутствие преобразование самих данных. Этот каст изменяет тип данных, но сама последовательность битов остаётся той же. Так как все указатели имеют одинаковый размер в байтах, то такое преобразование всегда возможно.

Чтобы преобразовать указатель `void*` обратно к тому типу, который соответствовал данным по сохранённому адресу следует использовать как раз `reinterpret_cast`.

```
void *ptr;

int i = 5;
ptr = &i;

int *i_ptr = reinterpret_cast<int *>(ptr);

std::cout << *i_ptr << std::endl;
```

Тип к которому нужно привести указатель указываем в угловых скобках

strict aliasing

С помощью `reinterpret_cast` можно создать ситуацию, когда адрес данных сохранён в двух указателях с разными связанными типами данных.

Тип, с которым создан объект. DynamicType

Другой тип, с которым осуществляется доступ AliasType

```
int i = 5;
void *ptr = &i;

int *i_ptr  = reinterpret_cast<int*>(ptr);
float *f_ptr = reinterpret_cast<float*>(ptr);
```

Доступ к данным посредством других типов приводит к неопределённому поведению, кроме случаев:

- Совпадает с DynamicType
- Является квалификацией DynamicType (например добавление const или volatile)
- Является similar к DynamicType (напримет `int const * const * *` и `int ***` similar)
- Является одним из типов агрегата (union) (ранее записанный с тем же типом или правильным alias)
- Базовый тип объекта, возможно с квалификаторами
- `char` , `unsigned char` или `std::byte` (C++17), возможно с квалификаторами
- signed и unsigned одного DynamicType (например `signed long` и `unsigned long`), возможно с квалификаторами

strict aliasing

```
void aliasing_not(int * p1, float *p2);
```

При компиляции такой функции оптимизатор может рассчитывать на то, что p1 и p2 сохраняют адреса на различные данные, что, в частности, можно использовать для группировки действий с p1 и p2 в одну длинную инструкцию, исполняемую параллельно. Если p1 и p2 будут указывать в одну область памяти, то результат будет непредсказуем.

```
void aliasing_yes(int * p1, unsigned const *p2);
```

В этом случае у нас действует правило о signed / unsigned типах одного DynamicType (в данном случае int) и добавлении квалификатора. Для оптимизатора это означает, что p1 и p2 могут смотреть в одну область памяти и для них агрессивные оптимизации применяться не будут.

std::byte

std::byte – это специальный тип данный, добавленный в C++17, который можно эффективно использовать вместо unsigned char для работы с байтами и битами. Основное отличие std::byte состоит в том, что он не представляет собой число и числовые операции с ним нельзя проводить, не приводя его явно к числу. Такая особенность снижает вероятность ошибок при работе с данными неизвестного происхождения. Тип объявлен в заголовочном файле `cstddef`.

Создаём из числа

Можно делать сдвиг

Арифметические операции не поддерживаются

Побитовые операции можно производить с байтами

```
353    std::byte b{15};   // 0x0F
354    b <<= 4;           // 0xF0
355    // b += 1;         // ошибка компиляции
356    b |= std::byte{1}; // 0xF0 | 0x01
357    std::cout << std::to_integer<unsigned>(b) << std::endl;
```

Преобразуем в число

Арифметика указателей

Сложение с числом, сравнение, ptrdiff_t, size_t

Сложение с числом (сдвиг)

При работе с данными может возникнуть необходимость переходить от одного элемента в памяти к другому. Например, если в памяти лежат несколько элементов типа `int` подряд, то можно быстро вычислять адрес каждого из них.



Сложение указателя с числом позволяет быстро производить такое вычисление.

```cpp
int i = 5;
int *i_ptr = &i;
std::cout << i_ptr << " " << (i_ptr + 1) << '\n';

std::byte b{1};
std::byte *b_ptr = &b;
std::cout << b_ptr << " " << (b_ptr + 1) << '\n';

double d = 1.0;
double *d_ptr = &d;
std::cout << d_ptr << " " << (d_ptr + 1) << '\n';
```

```
0x7ff7bb21f50c 0x7ff7bb21f510
```

+4

```
0x7ff7bb21f4ff 0x7ff7bb21f500
```

+1

```
0x7ff7bb21f4e8 0x7ff7bb21f4f0
```

+8

Сложение с числом (сдвиг)

При использовании сдвига нужно быть аккуратным! Хотя может показаться, что адрес – это просто число, но указатель более сложная абстракция. Сдвигать указатель можно только в пределах выделенной процессу памяти. Более того, если сдвиг указателя уходит далее, чем на единицу (размер одного элемента) за пределы памяти объекта, то результат такого сдвига не определён, т.е. результирующий адрес может оказаться произвольным.

```
377     int arr[4] = {10, 20, 30, 40};
378     int *ptr = &(a[0]);     // получаем адрес первого элемента
379
380     int *ptr3 = ptr + 3;    // Ok! Получаем адрес последнего элемента
381     int *ptr4 = ptr + 4;    // Ok! Первый элемент за концом данных, туда можно сдвинуть указатель
382     int a = *ptr;           // Ok! Можно прочитать данные из первого элемента
383     int b = *ptr3;          // Ok! Можно прочитать данные из последнего элемента
384     bool g =
385         (ptr3 + 1) == ptr4; // Ok! Результат определён и указатели должно быть равны
386
387     int *ptr5 = ptr + 5;     // НП! Результат сдвига неизвестен!
388     int c = *ptr4;           // НП! Нельзя читать или писать данные за пределы памяти объекта
389     int *ptr5b = ptr4 + 1;   // НП! Результат сдвига неизвестен!
390     bool s = ptr5 == ptr5b; // НП! Так как в 385 и в 387 неопределённое поведение, то результат сравнения не известен
```

Сравнение указателей

Указатели можно сравнивать между собой на совпадение == и != и на упорядоченность > , < , >= , <= . Важно при этом соблюдать правила сдвигов. Только указатели, которые определены в рамках одного объекта гарантируют верное поведение. Если сравниваются указатели за пределами объекта или принадлежащие разным объектам, то результат сравнения не определён.

```
394     int arr[4] = {10, 20, 30 ,40};
395     int *first_address = &(arr[0]);
396     int *second_address = &(arr[1]);
397     std::cout << std::boolalpha
398         << "f < s " << (first_address <  second_address) << '\n'
399         << "f > s " << (first_address > second_address) << '\n'
400         << "f == s " << (first_address == second_address) << '\n'
401         << "f != s " << (first_address != second_address) << '\n'
402         << "f + 1 == s " << (first_address + 1 == second_address) << '\n'
403         << "f + 2 < s " << (first_address + 2 < second_address) << '\n'
404         << "f + 2 > s " << (first_address + 2 > second_address) << '\n';
```

```
f < s true
f > s false
f == s false
f != s true
f + 1 == s true
f + 2 < s false
f + 2 > s true
```

Обход массива с использованием указателя

Получаем начальный адрес

Вычисляем адрес сразу за концом массива

```
int arr[10] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};

for (int *ptr = &(arr[0]), *past_the_end = ptr + 10; ptr != past_the_end; ++ptr)
    std::cout << *ptr << " | ";
```

На каждой итерации проверяем, что мы не попали в адрес за границей массива

Увеличиваем указатель на размер элемента `int` через операцию сдвига

Разность указателей

Указатели, относящиеся к одному объекту, можно вычитать. Разность будет вычислена в количестве сдвигов. Например, между двумя элементами типа int находится 4 байта (4 адреса), но разность составит единицу, так как от одного элемента до другого нужен сдвиг на один int. Результат представлен специальным переименованием типа `ptrdiff_t`.

```
int arr[10] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};

int *ptr_first = &(arr[0]); // первый элемент
int *ptr_last  = &(arr[9]); // последний элемент

ptrdiff_t diff_last_first = ptr_last - ptr_first;
ptrdiff_t diff_first_last = ptr_first - ptr_last;

std::cout << diff_last_first << " " << diff_first_last;
```

```
9 -9
```

Вычитать можно только указатели принадлежащие одному объекту, включая первый адрес за концом элемента. Если разность между адресами настолько велика, что сдвиг нельзя выразить в типе `ptrdiff_t`, то поведение не определено.

size_t

Также в заголовке cstddef определено специальное переименование типа size_t. Это переименование некоторого беззнакового типа, которое позволяет выразить размер в байтах любого допустимого на платформе объекта. Этот тип возвращается, например, при вызове оператора sizeof. Этот тип интересен тем, что с его помощью можно гарантированно выразить любое смещение. Т.е. для любых двух адресов ptr1 и ptr2 (ptr1 <= ptr2), принадлежащих одному объекту, существует такое число s из size_t, что ptr1 + s == ptr2. Его бывает удобно использовать для обхода массивов, особенно если мы хотим, например, скопировать один объект по байтам в другое место памяти.

```
std::cout
    << "size_t ["
        << std::numeric_limits<size_t>::min() << ", "
        << std::numeric_limits<size_t>::max() << "] " << sizeof(size_t) << "\n"
    << "ptrdiff_t ["
        << std::numeric_limits<ptrdiff_t>::min() << " , "
        << std::numeric_limits<ptrdiff_t>::max() << "] " << sizeof(ptrdiff_t) << "\n"
    << std::endl;
```

```
size_t [0, 18446744073709551615] 8
ptrdiff_t [-9223372036854775808 , 9223372036854775807] 8
```

Пример

```
453  void swap_4bytes(void *lha, void *rha) {
454      std::byte *lha_base = reinterpret_cast<std::byte*>(lha);
455      std::byte *rha_base = reinterpret_cast<std::byte*>(rha);
456
457      for (std::size_t offset = 0; offset != 4; ++offset) {
458          auto tmp = *(lha_base + offset);
459          *(lha_base + offset) = *(rha_base + offset);
460          *(rha_base + offset) = tmp;
461      }
462  }
```

```
int a = 5, b = 7;
swap_4bytes(&a, &b);
std::cout << "(a, b) " << a << " " << b << '\n';

char arr[4] = {'a', 'b', 'c', 'd'};
char brr[4] = {'1', '2', '3', '4'};
swap_4bytes(&(arr[0]), &(brr[0]));
std::cout << brr[0] << " " << brr[1] << " " << brr[2] << " " << brr[3] << '\n';

unsigned long l = 65535lu;
swap_4bytes(&l, reinterpret_cast<std::byte*>(&l) + 4);
std::cout << l << '\n';
```

```
(a, b) 7 5
a b c d
281470681743360
```